



*Logo Fast and Mudry - AI generated*

## **RAPPORT FAST & MUDRY**

Projet de POO

---

**Axel Schneider**

**Helder Ribeiro**

11 juin 2026

# Table des matières

- 1 – Introduction ..... 3
- 2 – Architecture du projet ..... 3
- 3 – Génération de la route et de la carte ..... 5
  - 3.1 – La ligne centrale ..... 5
  - 3.2 – De la géométrie à l'image ..... 6
  - 3.3 – Les biomes et les objets ..... 6
- 4 – La machine à états ..... 6
  - 4.1 – Les états et les évènements ..... 6
  - 4.2 – Le fonctionnement ..... 7
- 5 – Défi et problématique ..... 7
  - 5.1 – Rendu graphique ..... 7
  - 5.2 – Fonctionnement du rendu Mode 7 avec le GPU ..... 8
  - 5.3 – Affichage des objets et calcul de profondeur ..... 10

**Note :** Le rapport a été générée avec l'assistance d'une intelligence artificielle. Le contenu a été revu et validé par l'auteur avant son intégration au document final.

bonjour

## 1 – Introduction

Dans le cadre du projet de programmation orientée objet, nous avons développé Fast & Mudry, un jeu de course réalisé en Scala avec la bibliothèque LibGDX. Le joueur contrôle un véhicule évoluant sur un circuit généré dynamiquement, tout en devant éviter différents obstacles et gérer les contraintes liées à la conduite.

Au cours du développement, plusieurs choix techniques ont été réalisés afin d'améliorer l'organisation du code et le rendu visuel du jeu. L'une des évolutions les plus importantes a notamment été le remplacement d'un système de rendu basé sur une perspective calculée directement dans le code par une approche reposant sur le Mode 7 et l'utilisation de shaders graphiques. Cette évolution a conduit à une réorganisation de certains composants du moteur de rendu tout en conservant la logique métier du jeu.

Ce rapport présente l'architecture logicielle du projet, les principaux mécanismes mis en œuvre ainsi que les choix de conception retenus. Une attention particulière est portée à l'organisation des différents modules, aux interactions entre les composants et à l'application des concepts de programmation orientée objet.

## 2 – Architecture du projet

Le projet suit une architecture modulaire organisée autour de trois grands axes : la logique de jeu, la gestion des écrans et le rendu graphique. Cette séparation permet de garder le code plus lisible, plus maintenable et plus facile à étendre.

Les captures ci-dessous illustrent trois vues complémentaires du système : le flux d'application, le flux de données et le flux de rendu.

Le premier schéma, consacré au flux d'application, montre que le programme démarre par l'initialisation de la fenêtre principale, puis passe par une suite d'écrans successifs. L'application ne se limite donc pas à un simple programme linéaire : elle fonctionne comme une machine à états dans laquelle chaque écran représente une étape du jeu, comme le menu, le chargement, la partie, le garage ou l'écran de fin.

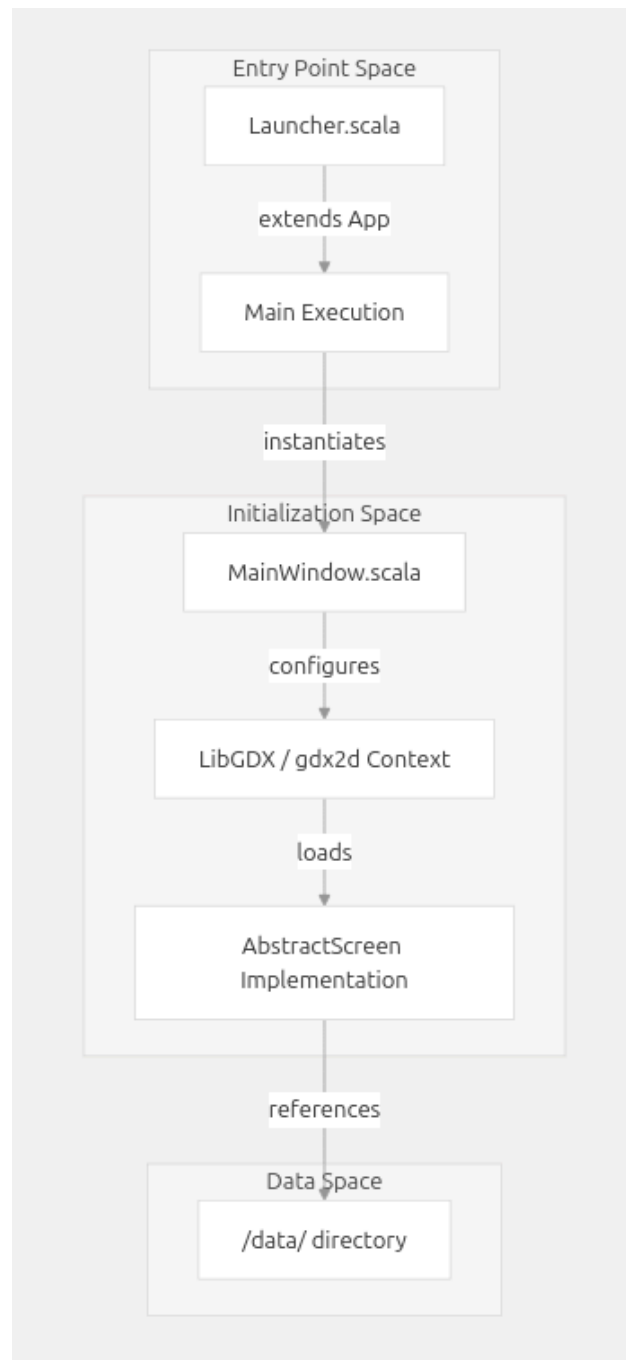


Figure 1 - Flux d'application du projet.

Le second schéma, centré sur le flux de données, met en évidence le rôle central du monde du jeu. Les entrées utilisateur, les assets graphiques et sonores, ainsi que les données de progression alimentent un état global partagé. À partir de cet état, la logique métier met à jour la position de la voiture, la progression du joueur, les événements du circuit et les éléments interactifs. Cette vue montre donc que le cœur du projet n'est pas dans l'affichage, mais dans la gestion cohérente des informations du jeu.

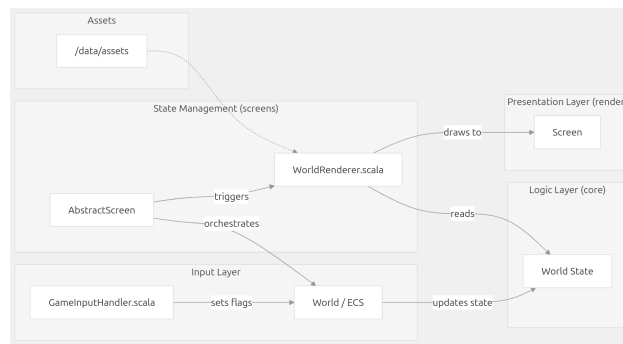


Figure 2 - Flux de données du projet.

Le troisième schéma, consacré au flux de rendu, explique comment cet état est ensuite transformé en image. Le module de rendu lit les informations du monde, puis les transmet aux différents sous-systèmes de dessin : fond, piste, objets, interface et HUD. Pour la piste, le rendu repose sur une projection pseudo-3D réalisée par un shader Mode 7. Cette étape montre que l’affichage n’est pas une simple copie de l’état du jeu, mais une interprétation visuelle de celui-ci.

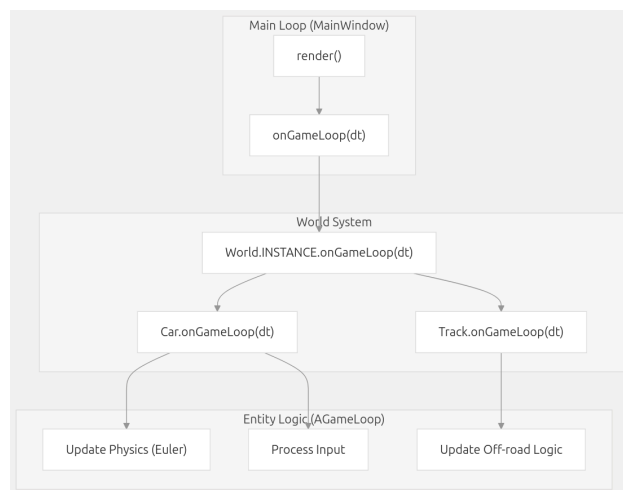


Figure 3 - Flux de rendu du projet.

Dans l’ensemble, ces trois vues sont cohérentes entre elles. L’application gère les états, le cœur du jeu met à jour les données, et le moteur graphique transforme ces données en image. C’est cette séparation entre logique, données et rendu qui donne au projet son architecture modulaire et compréhensible.

### 3 – Génération de la route et de la carte

Un des objectifs du projet était que chaque partie soit un peu différente. Plutôt que de dessiner les circuits à la main, nous avons choisi de générer la route de manière procédurale au début de chaque journée de jeu. À chaque chargement, le joueur découvre donc un circuit qu’il n’a jamais vu.

#### 3.1 – La ligne centrale

Tout part de la classe TrackGeometry, qui construit la ligne centrale de la route. Elle reçoit un point de départ, un point d’arrivée et un nombre de points de contrôle (20 dans notre cas). Les points sont d’abord répartis régulièrement entre le départ et l’arrivée, puis chacun reçoit un décalage aléatoire vertical et horizontal, sauf le premier et le dernier qui restent fixes pour garder un départ et une arrivée propre. À ce stade, la route ressemble à une suite de segments anguleux, ce qui n’est pas très agréable à conduire. Les points sont donc ensuite lissés avec une spline de Catmull-Rom, qui interpole 30 sous-points entre chaque paire de points de contrôle. Le résultat est une courbe fluide avec des virages naturels.

```
1 val yNoise = ((Math.random - 0.5) * 2.0 * randomHeight).toFloat
```

```

2  val h: Float = start.y + height * i + yNoise
3  val xNoise = if (i == 0 || i == nPoints - 1) 0f
4              else ((Math.random - 0.5) * 2.0 * width * 0.4).toFloat
5  val p: Vector2 = new Vector2(start.x + width * i + xNoise, h)

```

Listing 1 - Génération des points de contrôle aléatoires de la route

Cette ligne centrale sert ensuite de référence pour tout le reste. La géométrie expose une méthode qui calcule la distance entre un point quelconque et la ligne, ce qui permet de savoir si la voiture est sur la route ou non. Comme ce calcul est appelé énormément de fois, chaque segment possède une boîte englobante qui permet d'ignorer rapidement les segments trop éloignés. La ligne d'arrivée est simplement définie comme un cercle autour du dernier point de la courbe.

### 3.2 – De la géométrie à l'image

La carte affichée à l'écran est en réalité une grande texture générée par TrackTexture. L'objet parcourt chaque pixel de l'image et calcule sa distance à la ligne centrale. En fonction de cette distance, le pixel devient soit la ligne du milieu de la route, soit la route elle-même, soit le bas-côté, soit le terrain hors-piste. Les pixels proches du point d'arrivée sont dessinés en noir pour matérialiser la ligne d'arrivée.

Pour éviter d'avoir des surfaces complètement uniformes, les couleurs ne sont pas fixes. Chaque biome fournit une couleur claire et une couleur foncée pour chaque type de surface, et un bruit de Perlin vient mélanger les deux. Ça donne des textures avec des couleurs un peu différentes d'un pixel à l'autre, ce qui rend le sol beaucoup plus vivant. Le même principe est utilisé pour faire varier légèrement la largeur du bas-côté, comme ça le bord de la route n'est pas une ligne parfaite.

### 3.3 – Les biomes et les objets

Chaque journée du jeu possède son propre biome : la forêt, le désert et la neige. Le biome définit les couleurs de la carte, la musique, le décor en arrière-plan et aussi la physique quand on sort de la route. C'est également lui qui place les objets sur la carte, comme les arbres, les rochers ou les cactus. Pour chaque objet, une position est tirée aléatoirement sur la carte, et on recommence le tirage tant que la position ne convient pas, par exemple pour éviter qu'un arbre se retrouve au milieu de la route. Le bâtiment de la HES, lui, est toujours posé exactement sur le point d'arrivée.

Comme la génération de la texture est assez lourde (chaque pixel demande un calcul de distance), elle est lancée dans un thread séparé pendant que l'écran de chargement affiche une petite animation. Ça nous a permis d'éviter de bloquer la fenêtre du jeu pendant la génération. Une fois le calcul terminé, la texture est envoyée au GPU, la voiture est placée sur le premier point de la ligne centrale et orientée vers le deuxième, et la partie peut commencer.

## 4 – La machine à états

Le jeu est composé de plusieurs phases bien distinctes : le menu, le chargement, la course, les cinématiques, le quiz, le garage... Au début, les changements d'écrans étaient faits un peu à la main, mais on s'est vite rendu compte que ça devenait difficile de savoir qui avait le droit de passer où. Pour mettre de l'ordre, nous avons centralisé toute cette logique dans une machine à état.

### 4.1 – Les états et les événements

Chaque phase du jeu est représentée par un état, défini dans le trait scellé GameState. On y retrouve par exemple Menu, Loading, Playing, Quiz, Garage, Dead ou encore FinalCinematic. Certains états transportent en plus le jour courant de la partie (Day1, Day2 ou Day3), ce qui permet de savoir où le joueur en est dans sa progression sans avoir besoin d'une variable globale à côté.

Les transitions sont déclenchées par des événements, comme StartGame, MapLoaded, FinishLineCrossed ou CarBroke. L'idée centrale est que chaque état décide lui-même de sa transition : il implémente

une méthode `next` qui reçoit un évènement et retourne le prochain état. Si l'évènement ne le concerne pas, l'état se retourne simplement lui-même et rien ne se passe.

```

1  final case class Playing(day: Day) extends GameState {
2      def next(event: GameEvent): GameState = {
3          event match {
4              case FinishLineCrossed if day.next.isEmpty => FinalCinematic
5              case FinishLineCrossed => EndDayCinematic(day)
6              case CarBroke => Dead
7              case _ => this
8          }
9      }
10 }

```

Listing 2 - L'état Playing et ses transitions possibles

## 4.2 – Le fonctionnement

L'objet `GameStateMachine` garde en mémoire l'état courant. Quand un évènement arrive, il demande à l'état courant quel est le prochain état. Si l'état change, la machine active l'écran qui correspond via le gestionnaire d'écrans : l'état Menu affiche l'écran du menu, l'état Playing affiche l'écran de jeu, et ainsi de suite. Si l'état ne change pas, l'évènement est simplement ignoré. C'est un gros avantage : un évènement envoyé au mauvais moment ne peut pas casser le jeu ni provoquer des comportements bizarres.

Les évènements sont envoyés depuis un peu partout dans le code. Les boutons du menu envoient `StartGame` ou `OpenCarSelector`, l'écran de chargement envoie `MapLoaded` quand la carte est prête, la piste envoie `FinishLineCrossed` quand la voiture franchit la ligne d'arrivée ou `CarBroke` quand le véhicule est cassé, et les cinématiques envoient leur évènement de fin une fois leur durée écoulée. Aucun de ces endroits n'a besoin de connaître l'écran suivant, ils signalent juste ce qui vient de se passer.

Une partie complète suit donc un cycle assez naturel : depuis le menu, on passe au chargement puis à la course du premier jour. Quand la ligne d'arrivée est franchie, une cinématique de fin de journée se lance, suivie du quiz puis du garage où le joueur peut réparer sa voiture. Une cinématique de début de journée enchaîne ensuite sur le chargement du jour suivant. Au troisième jour, franchir la ligne d'arrivée déclenche la cinématique finale qui ramène au menu. Et si la voiture casse en route, peu importe le jour, on passe directement à l'écran de mort.

Ce découpage nous a beaucoup aidé sur la fin du projet : pour rajouter une nouvelle phase, comme le sélecteur de voiture, il suffisait de rajouter un état, ses évènements et son écran, sans toucher au restant de la logique.

## 5 – Défi et problématique

### 5.1 – Rendu graphique

L'un des principaux défis rencontrés durant le développement de *Fast & Mudry* concernait le rendu graphique de la piste. Les premières versions du projet utilisaient un affichage de type arcade reposant sur une simulation de perspective calculée directement dans le code. Cette approche permettait de représenter une route et ses virages avec un coût de développement relativement faible, mais elle présentait plusieurs limitations en termes de lisibilité et de perception de l'environnement.

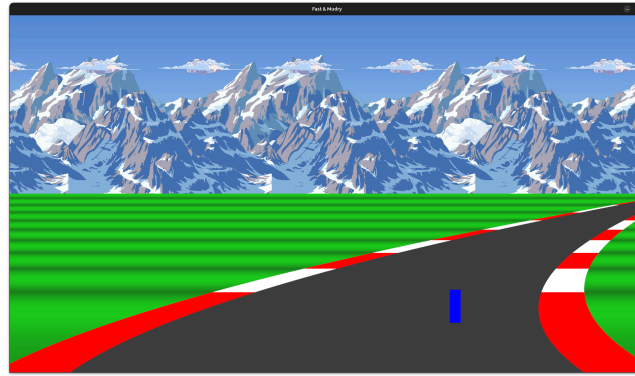


Figure 4 - Première version du rendu

En pratique, les virages étaient difficiles à anticiper pour le joueur. La représentation de la piste manquait de profondeur et les changements de direction apparaissaient souvent de manière abrupte. Cette limitation avait un impact direct sur le gameplay : le joueur disposait de peu d'informations sur ce qui se trouvait au-delà de la portion immédiatement visible de la route. Il devenait alors difficile d'anticiper les obstacles, de préparer une trajectoire adaptée ou d'évaluer la géométrie du circuit à moyen terme.

Afin d'améliorer cette situation, le système de rendu a été repensé autour d'une approche de type Mode 7. Cette technique consiste à projeter une texture représentant le circuit de manière à créer une illusion de profondeur tout en conservant un environnement essentiellement bidimensionnel. Contrairement à l'ancien rendu, cette approche offre une meilleure perception de l'espace et permet d'afficher une portion beaucoup plus importante du circuit devant le véhicule.



Figure 5 - Rendu avec mode 7

Cette évolution a considérablement amélioré la lisibilité du jeu. Le joueur peut désormais distinguer les virages à venir, observer les obstacles situés plus loin sur la piste et repérer certains éléments importants du décor, comme la HES qui matérialise la ligne d'arrivée. La visibilité accrue permet une conduite plus naturelle, fondée davantage sur l'anticipation que sur la réaction immédiate aux événements apparaissant à l'écran.

## 5.2 – Fonctionnement du rendu Mode 7 avec le GPU

Dans l'architecture actuelle, la responsabilité de créer l'image affichée est donnée à TrackRenderer. Cet objet joue le rôle d'intermédiaire : il récupère la texture du circuit, place une caméra virtuelle selon la position du véhicule, envoie ensuite plusieurs paramètres au shader et lance le rendu. Cette séparation est pratique, car la logique du jeu reste dans les classes métier, tandis que la partie graphique se concentre sur la transformation de cette information en image.

Le composant Mode7.scala ne dessine pas directement l'écran. Il sert surtout à définir la configuration du shader : les noms des paramètres, les chemins vers le programme graphique et les valeurs par défaut. Ces paramètres sont ensuite transmis au shader par TrackRenderer. Le shader reçoit alors des informations telles que la résolution de l'écran, la position de la caméra, son angle, l'axe de rotation, la



distance du plan de projection et le facteur d'échelle de la texture. Cette organisation permet de garder la projection dans un module spécialisé, indépendant de la physique de conduite ou de la gestion du jeu.

La position et l'orientation de la voiture influencent directement le rendu. À chaque image, TrackRendre met à jour la caméra à partir des coordonnées du véhicule et de sa rotation. Ces valeurs sont envoyées au shader, qui recalculera alors l'affichage du circuit pour chaque pixel. Ainsi, quand la voiture tourne, la texture du circuit est réinterprétée sous un autre angle, ce qui donne l'illusion d'un déplacement sur une route en perspective. On peut donc voir le véhicule comme une entrée de rendu, au même titre que la taille de l'écran ou la résolution de la texture.

```

1 cameraPosition.y = World.INSTANCE.CAR.Coordinates.y
2 cameraPosition.x = World.INSTANCE.CAR.Coordinates.x
3 cameraAngle = World.INSTANCE.CAR.Rotation
4
5 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.POSITION, cameraPosition)
6 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.ANGLE, cameraAngle)

```

Listing 3 - Transmission des informations de déplacement au shader

Dans le shader, la transformation part des coordonnées du fragment, c'est-à-dire des pixels à afficher à l'écran. Pour chaque pixel, le shader construit un rayon de vue dans un espace virtuel, puis applique une rotation selon l'angle de la caméra. Cette étape permet de transformer un point de l'écran en une direction de projection, puis de retrouver la position correspondante dans la texture du circuit. Le calcul combine ensuite la position de la caméra, la distance du plan de projection et le facteur de mise à l'échelle pour obtenir une coordonnée de texture comprise entre 0 et 1.

$$\begin{aligned}
 r &= (x_s, y_s, d) \\
 r' &= R(a)r \\
 p &= c + (r'_x, r'_y) * \begin{pmatrix} c_z \\ r_z \end{pmatrix} \\
 t &= (p - o) * f
 \end{aligned}$$

auto 6 - Formulation générale de la projection du fragment

Cette écriture résume le calcul effectué par le shader : un fragment, défini par ses coordonnées écran  $x_s$  et  $y_s$ , correspond d'abord à un rayon  $r$  dans un espace virtuel. Ce rayon est ensuite tourné selon l'angle  $a$  de la caméra, puis il est projeté sur un plan de référence pour obtenir une position  $p$  dans le repère de la carte. La coordonnée de texture  $t$  est enfin obtenue en appliquant une mise à l'échelle à partir de l'origine  $o$  et du facteur  $f$ . Lorsque cette coordonnée se trouve sur le circuit, la couleur correspondant à cette zone est lue dans la texture et affichée à l'écran. Le mécanisme est donc un problème de géométrie simple, mais appliqué à chaque pixel par le GPU.

```

1 vec3 pixelRay;
2 pixelRay.xz = (gl_FragCoord.xy / resolution.xy) * vec2(1.0, -1.0) + vec2(-0.5,
3 0.5);
4 pixelRay.y = screenPlanDistance;
5 pixelRay *= rotationMatrix(cameraAxis, cameraAngle);
6
7 vec2 hitPosition = cameraPosition.xy + pixelRay.xy * (cameraPosition.z /
8 pixelRay.z) + vec2(0.5, 0);
9 hitPosition = (hitPosition - mapOrigin) * renderingFactor;
10 gl_FragColor = texture2D(backbuffer, hitPosition);

```

Listing 4 - Projection du fragment vers une coordonnée de texture

Le GPU est particulièrement adapté à ce type de traitement, car il peut effectuer des calculs sur des millions de pixels en parallèle. Contrairement à un traitement séquentiel sur le CPU, le shader traite simultanément tous les pixels de l'image. Cette parallélisation est très utile ici, puisque chaque pixel

est calculé à partir de la même logique de projection. Le processeur graphique prend donc en charge la transformation géométrique et la lecture de la texture, tandis que le CPU peut se concentrer sur la logique du jeu.

Le rendu repose sur une projection de la texture du circuit sur une surface virtuelle. Cette méthode est simple et efficace pour un jeu de course en 2D/2.5D, mais elle dépend beaucoup de la résolution de la texture, du réglage de la caméra et de l'alignement entre la piste et les objets décoratifs. Le shader doit aussi être réglé avec soin pour éviter les artefacts aux bords du circuit ou dans les zones où la projection devient instable. C'est un compromis adapté au projet : obtenir un rendu fluide sans ajouter une complexité de calcul trop importante.

### 5.3 – Affichage des objets et calcul de profondeur

La même logique de projection est utilisée pour afficher les objets de jeu, tels que les obstacles ou les éléments décoratifs, au-dessus du circuit. Chaque item possède une position dans le monde, exprimée dans les coordonnées du circuit, et il est projeté dans l'espace de l'écran à partir de la position de la caméra et de son orientation. Cette transformation repose sur le calcul d'un vecteur différentiel entre la position de l'objet et la caméra, puis sur une rotation de ce vecteur selon l'angle du véhicule. Le résultat de cette opération permet de déterminer si l'objet se trouve devant ou derrière la caméra virtuelle. Si la composante de profondeur est négative ou trop faible, l'objet n'est pas projeté, ce qui évite son affichage lorsqu'il se trouve hors du champ de vision ou derrière le plan de vue.

Lorsque l'objet est visible, sa coordonnée à l'écran est obtenue par une projection perspective simple. La position horizontale est calculée à partir du rapport entre la distance latérale de l'objet et sa profondeur, tandis que la position verticale dépend à la fois de la hauteur de la caméra et de l'inclinaison du point de vue. Le facteur d'échelle est lui aussi déterminé par la profondeur selon une relation de la forme suivante.

$$s = \frac{d_p}{p_y}$$

auto 7 - Échelle de l'objet calculée à partir de sa profondeur relative

Cette relation garantit une impression cohérente de profondeur et permet de conserver une hiérarchie visuelle naturelle entre les objets proches et lointains.

Dans l'implémentation, ce calcul est centralisé dans `ItemsRenderer`. La méthode de projection construit un résultat composé de coordonnées d'affichage, d'un facteur d'échelle et d'une distance, puis trie les objets selon cette distance avant de les dessiner. L'ordre de rendu est ainsi déterminé par la profondeur perçue selon une priorité de traitement représentée par la relation suivante.

$$x_e = w * \left( 0.5 + \frac{d_p * p_x}{p_y} \right)$$

auto 8 - Coordonnée horizontale de projection sur l'écran

$$y_e = h * \left( 0.5 + t - \frac{d_p * z_c}{p_y} \right)$$

auto 9 - Coordonnée verticale de projection sur l'écran

Cette stratégie est importante, car elle évite que les éléments du décor ou les obstacles se superposent de manière incohérente et contribue à préserver la lisibilité de la scène.

```

1  val differentialVector = new Vector2(position.x, position.y)
2  differentialVector.sub(cameraPosition.x + 0.5f, cameraPosition.y)
3
4  val primeVector = new Vector2(
5      differentialVector.x * cosAlpha - differentialVector.y * sinAlpha,
6      differentialVector.x * sinAlpha + differentialVector.y * cosAlpha
7  )
8
9  if (primeVector.y ≤ 0.01f) return None
10

```

```
11 val screenX = screenW * (0.5f + (screenPlanDistance * primeVector.x) /  
primeVector.y)  
12 val screenY = screenH * (0.5f + pitch - (screenPlanDistance * cameraPosition.z) /  
primeVector.y)  
13  
14 val scale = screenPlanDistance / primeVector.y
```

Listing 5 - Projection d'un item dans l'espace de l'écran

Le flux de données peut être résumé de la manière suivante : état du véhicule → uniforms transmis par TrackRenderer → shader GPU → échantillonnage dans la texture du circuit → image finale affichée à l'écran. Pour les items, ce flux est complété par une projection supplémentaire, qui transforme leur position dans le monde en coordonnée d'écran et en échelle visuelle.